Approval-Based Elections in the 1D Euclidean Domain

Jan Sznajd

Supervisor: dr hab. inż. Piotr Faliszewski

Institute of Computer Science
AGH University of Science and Technology

This dissertation is submitted for the degree of
Master of Science

May 2021

Contents

1 Introduction **1**

2 Preliminaries **2**

3 One Dimensional Euclidean Domain **4**

3.1 Preference Restrictions 4

3.2 Euclidean Preference Restrictions 4

4 Voter/Candidate Range Profile Detection **6**

4.1 Profile Structure Detection 6

4.2 Integer Linear Program 7

4.3 C1P Detection — ILP vs SAT Performance Comparison 9

5 Voter/Candidate Range Profiles **10**

5.1 Domain analysis 10

5.2 How many VCR profiles are not VR neither CR? 10

5.2.1 Experiment 1 — Small Profiles 10

5.2.2 Experiment 2 — large profiles 11

5.3 Transforming VCR (NCOP) Profile into VR or CR 13

6 Control Problem **14**

6.1 Overview 14

6.1.1 AV-CC-DV for Single Peaked (VR = CI = row C1P = $r_c = 0$) . . . 14

6.1.2 AV-CC-DV for Candidate Range = VI = column C1P = $r_v = 0$ . . . 15

6.2 AV-CC-DV for the VCR profiles 15

6.2.1 Sketch Proof - CR case 16

6.2.2 Sketch Proof - VR case 17

6.2.3 Sketch Proof - VCR (not C1P) case 19

6.3 AV-CC-AV for the VCR profiles 20

7 Evaluation **21**
7.1 Conclusions 21
7.2 Future Work 21

Bibliography **22**

Chapter 1

Introduction

Elections are ubiquitous element of our society in one form or another. For example choosing members of parliament or a winner of a competition. In a day-to-day activities we need to decide on a meeting time or select recommended movies in a streaming service. Each of these choices may require different type of election and scoring rule, based on the underlying properties of the choice (i.e. voters should be represented proportionally by members of a government)

Reseach objectives

In this work I am going to focus on the *Approval Elections*, that is where each voter provides a subset of candidates that he approves of and disapproves the rest. In general many social choice problems are known to be computationally hard. However they become tractable under some restrictions on the structure of the preferences. Elkind and Lackner (2015) explored well known restrictions in the setting of approval preferences, introduced several new one-dimensional domain restrictions such as Dichotomous Euclidean (sprawdzic kiedy/kto wprowadził pojęcie - spatial pref, multidim unfolding) and derivatives (czy to dobre okreslenie na VR CR?). Faliszewski (2021) generalized this model for the N-dimensional euclidean space calling it Voter Candidate Range and explored two-dimensional case.

The goal of this work is to explore properties of the one-dimensional model. Search for the profile detection polynomial complexity algorithm. I will try as well to extend solution of **?** for computing MAV score with polynomial complexity to the VCR model.

Contributions

I provide linear programming preference detection algorithm and ...

Chapter 2

Preliminaries

In this chapter, we introduce the definitions and notations used in this work.

Approval Elections

Election is a pair $E = (C, V)$, where $C = \{c_1, ..., c_m\}$ is a set of candidates and $V = \{v_1, ..., v_n\}$ is a set of voters. Each voter $v \in V$ approves a subset of the candidates and disapproves the remaining candidates. I will be referring to it as a *approval set* and for each voter $v \in V$ denote it as $A(v)$, when using A just by itself I refer to all approval sets in the given election (preferences).

Committee Selection Rules

Voting rules are algorithms used to select winners (committee) of a given election. Each rule has different properties and must be selected accordingly to the goal of the election. For the detailed overview of multiwinner voting rules I point to Faliszewski et al. (2017).

WIP: Jakie jeszcze tematy/pojęcia tu opisać

Integer Linear Program

Integer linear programming (ILP) is a common approach to solve NP-hard problems, furthermore it can be used effectively for other problems as well. (WIP: czy zdanie od furthermore ... jest potrzebne) To define a computational problem as an integer linear program, we have to create a system of linear inequalities (constraints) (WIP: Sprawdzic czy na pewno inequalities == constraints) and optionally a linear function to optimize, alternatively we can just check feasibility of the constraints. Next to obtain a solution we have to use an optimization solver. Popular choices are Gurobi or CPLEX which are state-of-the-art general purpose solvers.

(WIP: Czy przenieść do Preliminaries + opisać rodzaje zmiennych w ILP, etc. ?)

Chapter 3

One Dimensional Euclidean Domain

3.1 Preference Restrictions

(WIP: Motywacja, po co restrykcje, czy moge powtorzyc Introduction?)

Many social choice problems such as winner determination, or control problem are known to be computationally hard. To deal with it we can use approximate or parametrized algorithms (WIP: Podać przykłady metod / cytacje). The alternative approach is to introduce restrictions on the preference domains. (WIP: Cytacje research'u) In the last few years this area of research advanced a lot, providing multiple efficient algorithms for the popular committee selection rules, and other problems.

3.2 Euclidean Preference Restrictions

Voter Candidate Range

Given an election $E = (C, V)$, we say that the voters have *1D-Euclidean* preferences if for every agent (voter or candidate) there exists a point x_a and nonnegative real value radius r_a such that: $\forall\, c \in C\ \forall\, v \in V : c \in A(v) \Leftrightarrow \sqrt{(x_c - x_v)^2} \leq r_c + r_v$

We can easily think of it as: the point x_a represents ideal position in the 1-dimensional space of opinions. The radius can be seen as candidate's charisma or voter's willingness to compromise.

Candidate Range

We say that P satisfies CR if candidates in C can be ordered so that each of the sets v_i forms an interval of that ordering. CR is special case of VCR, where we require all the voters to have radii equal to zero

Voter Range

We say that P satisfies VR if the voters in V can be reordered so that for every candidate c_i the voters that approve c form an interval of that ordering. VR is a special case of VCR, where we require all the candidates to have radii equal to zero

Voter Interval and Candidate Interval

Faliszewski (2021) showed that CR and VR are equivalent to respectively VI and CI introduced by Elkind and Lackner (2015).

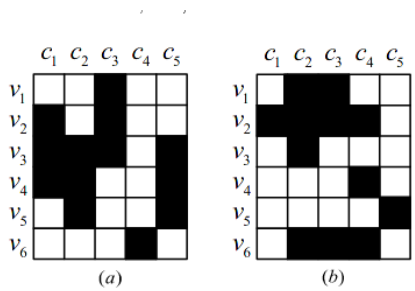

Figure 3.1 CR and VR Temporary image

Larger Domain

(WIP: Zmienić na mój przykład i opisać dlaczego nie vr i nie cr)

The VCR model has larger domain than the VR and CR combined. For example, election with candidate set $C = \{a,b,c,d\}$ and voters with approval sets $\{a,b\}$, $\{b,c\}$, $\{b,d\}$, and $\{a,b,c,d\}$ is VCR, but neither VR nor CR.

Chapter 4

Voter/Candidate Range Profile Detection

In this chapter we present our algorithms for deciding if a given approval profile has the VCR property.

4.1 Profile Structure Detection

To take advantage of the restrictions defined in Section 3, we need to design detection algorithms. They can be used to check whether given election profiles satisfy particular structural properties. For example, to check the VR (CR) property it is enough to check for the *consecutive ones property* in the preference matrix.

Definition 1. *A binary matrix (i.e., a matrix consisting of zeros and ones) has the consecutive ones property (C1P) if its columns be permuted in such a way that for each row the ones are consecutive, i.e., the 1s form an interval.*

In this work we refer to the term *C1P* more broadly, using it interchangeably, for an analogous scenario where we permute the rows and look for the consecutive 1s in every column in a given matrix. Booth and Lueker (1976) provided a linear-time C1P detection algorithm based on the PQ-Tree data structure.

Proposition 1 (Elkind and Lackner (2015))**.** *Deciding whether an approval profile satisfies VR or CR is possible in $O(m \cdot n)$ time.*

Proof. Let $C = \{c_1, ..., c_m\}$ and $V = \{v_1, ..., v_n\}$, we create a $n \times m$ matrix A (a_{ij}): for each $i \in [n]$ and $j \in [m]$: $a_{ij} = 1$ if $c_j \in A(v_i)$ and 0 otherwise. We can see that for the VR property, permuting the columns of matrix A so that ones form an interval in each row is equivalent to permuting candidates so that the set of candidates approved by each voter forms an interval. For the CR case, we just need to transpose the matrix created for the VR. □

Our focus lies on the one dimensional VCR property. Having preference matrix as input we have to provide for each agent (voter/candidate) a point on the real line and a nonnegative real value representing radius, so that these parameters satisfy the VCR property.

4.2 Integer Linear Program

An algorithm based on ILP solving is an excellent choice for the first attempt at designing a detection algorithm. It provides an efficient and relatively simple solution. Furthermore, it can be used as a fast benchmark for validating future solutions. Below we provide an ILP for the VCR profile recognition, together with additional preliminary steps.

Transforming the Definition of VCR into Linear Constraints

Before formulating an ILP for detecting VCR profiles, we have to redefine the definition of VCR in order to make it appropriate for an ILP solver. The definition is based on the euclidean distance which cannot be directly represented in an ILP. Equation 4.2 shows transformation of the VCR definition into linear inequalities.

(WIP: Czy powtarzam *VCR definition* za często?)

As a short reminder from Section 3.2, for the candidates C and voters V, 1D-VCR is defined as:

$$\forall\, c \in C\, \forall\, v \in V : c \in A(v) \Leftrightarrow |x_c - x_v| \leq r_c + r_v \tag{4.1}$$

Where x_a and r_a represent agent's point on the real line and their radius respectively.

(WIP: W jaki sposób mogę zwięźle, bez ponownego wpropadzania wszystkich oznaczeń, przypomieć definicję? O ile w ogóle jest to potrzebne...)

$$\begin{aligned} &|x_c - x_v| \leq r_c + r_v \\ &\iff -(r_c + r_v) \leq x_c - x_v \leq r_c + r_v \\ &\iff x_c - x_v \leq r_c + r_v \;\; \vee \;\; -(x_c - x_v) \leq r_c + r_v \end{aligned} \tag{4.2}$$

(WIP: Poprawić formatowanie, jak?)

The constraint obtained from the equation 4.2 is not suitable yet for an ILP because it contains logical *OR* condition which again cannot be directly represented in an ILP. To transform it into linear form, we apply commonly called *BigM* technique, as shown below.

$$
\begin{aligned}
x_c - x_v &\leq r_c + r_v + M(1 - z_1) \\
-(x_c - x_v) &\leq r_c + r_v + M(1 - z_2) \\
z_1 + z_2 &\geq 1 \\
z_1, z_2 &\in \{0, 1\}
\end{aligned}
\tag{4.3}
$$

First, we have to introduce a large constant M (larger than $max(|x|)$) and two binary variables z_1 and z_2. Now if any of the z_i is 0 then the corresponding inequality will be true, and the additional constraint $(z_1 + z_2 \geq 1)$ guarantees that at least one of the z_i will be 1.

(WIP: Trochę namieszane wytłumaczenie, bo tak naprawdę problem z *OR* jest tylko dla wariantu if $c \notin A(v)$, czyli $x_c - x_v + M(1 - z) \geq r_c + r_v + 1$ ...)

The BigM technique is common enough, for most solvers to provide additional built-in mechanisms for applying it. Using them is usually recommended, as it allows a solver for better internal optimization. In Gurobi solver we use *indicator constraint*, which allows one to create versatile constraints in form of: $z = f \rightarrow a^T x \leq b$. The constraint states that if the binary indicator variable z is equal to f, where $f \in \{0, 1\}$, then the linear constraint $a^T x \leq b$, where a and b are data coefficients vectors and x represents the decision variables, should hold. On the other hand, if $z = 1 - f$, the linear constraint may be violated.

(WIP: BigM jest używane do reprezentacji różnych 'funkcji', nie tylko *OR*, np. *max*, *abs*, etc., czy też o tym wspomnieć?)

Detection Algorithm

Final formulation of the ILP for recognising VCR property in a given election profile is shown in Figure 4.4. The program accepts preference matrix A as input, and if solution exists it outputs every agent's position x_a and radius r_a. Additionally, we use two binary variables p and q, required by the indicator constraints.

$$
\begin{aligned}
&\textbf{check feasibility}: \\
&\quad \textbf{subject to}: \\
&\qquad x_a \in \mathbb{R} && \forall\, \mathrm{a} \in C \cup V \\
&\qquad r_a \in \mathbb{R}^{0+} && \forall\, \mathrm{a} \in C \cup V \\
&\qquad p_{cv} \in \{0,1\} && \forall\, \mathrm{c} \in \mathrm{C}\, \forall\, \mathrm{v} \in V \\
&\qquad q_{cv} \in \{0,1\} && \forall\, \mathrm{c} \in \mathrm{C}\, \forall\, \mathrm{v} \in V \\
&\quad \text{if } c \in A(v): \\
&\qquad x_c - x_v \leq r_c + r_v \\
&\qquad -(x_c - x_v) \leq r_c + r_v \\
&\quad \text{if } c \notin A(v): \\
&\qquad p_{cv} == 1 \rightarrow x_c - x_v \geq r_c + r_v + 1 \\
&\qquad q_{cv} == 1 \rightarrow -(x_c - x_v) \geq r_c + r_v + 1 \\
&\qquad p_{cv} + q_{cv} >= 1
\end{aligned}
\tag{4.4}
$$

4.3 C1P Detection — ILP vs SAT Performance Comparison

Chapter 5

Voter/Candidate Range Profiles

In this section we explore the VCR profiles domain and describe some of its properties. The main goal is to compare the size of the domain with other well known domains such as VR and CR.

5.1 Domain analysis

WIP: Najpierw Napisać dlaczego / po co to robimy. Dopiero potem co / w jaki sposób. Chcemy sprawdzić czy warto wymyślać algorytmy VCR i czy doemna jest znacząco większa od VR / CR

We have performed several experiments, in each of them we check how many profiles have the VCR property but not VR neither CR. For the first one we focus on the small size elections, and for the second experiment we employ several strategies to generate random, larger profiles with the VCR property.

5.2 How many VCR profiles are not VR neither CR?

5.2.1 Experiment 1 — Small Profiles

For the small profiles we did a complete search of the domain. That is we generated every possible profile of a given size and checked if it has the VCR property. We checked 6 dimensions of the elections with the following sizes, having M candidates and N voters, $(M,N) \in \{(3,3),(4,4),(5,5),(4,6),(6,4)\}$. The results as seen in Figure 5.1 show that VCR domain is relatively large, however only limited number of profiles are neither VR nor CR.

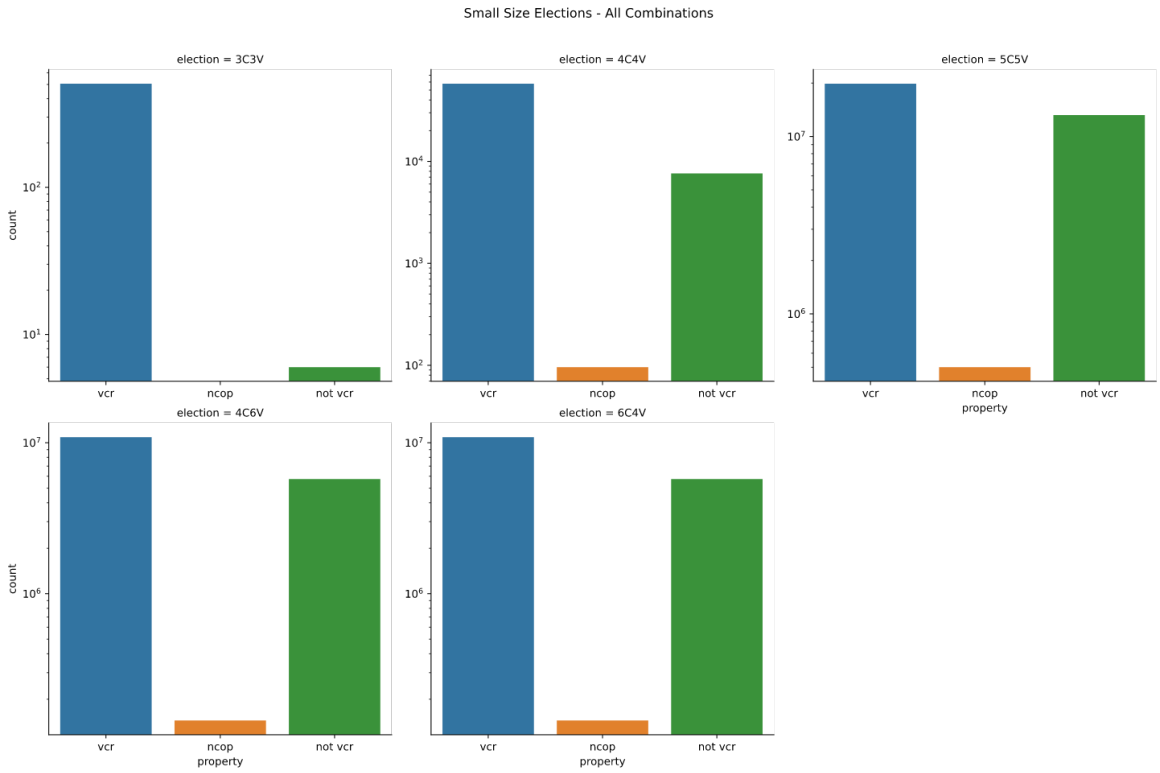

Figure 5.1 Temporary Image - Aggregated Statistics For Small Profiles

5.2.2 Experiment 2 — large profiles

For the larger size of the profiles we generated a suitable number of random VCR profiles and again checked how many of them are neither CR nor VR (*NCOP*), and additionally, VR and CR (*COP*), VR and not CR, CR and not VR. We employed several strategies for generating the profiles. The first parameterized category is a statistical distribution used to pick the points representing voters and candidates. The second one is the choice of radius parameter. We describe the categories in detail below. For each parameter pair we generated 1000 elections with 20 candidates and 20 voters, then we repeated the experiment for the profiles of 40 candidates and 40 voters.

Position parameter

We consider 4 ways of generating the points, representing the positions of the candidates and the voters.

1. The *Uniform Candidates, Uniform Voters* model, where the points for both, the candidates and the voters are selected uniformly at random from the $[-10, 10]$ range.

2. The *Asymmetric Gauss Candidates, Asymmetric Gauss Voters* model, where we take 70% of points from a Gaussian distribution with mean value -3 and standard deviation 1.8, and the remaining 30% come from a Gaussian with mean 3 and the same standard deviation. This model is inspired by the 2 dimensional model from the work of Faliszewski (2021). The goal is to simulate the society with a large majority, and a minority with the opposite opinion.

3. The *Uniform Candidates, Asymmetric Gauss Voters* model, it is a combination of the previous two models, where the candidate's points are selected uniformly at random, and for the voters we use asymmetric Gaussian model. We use the same parameters for both of the distributions.

4. The *Asymmetric Gauss Candidates, Uniform Voters* model, it is mirror version of the previous model, with the distributions swapped.

Radius parameter

In this category we consider 5 ways of generating the radii. We split the methods between the candidates and the voters, similarly to picking the positions.

1. The *Gauss Candidates, Gauss Voters* model, where we choose the radii from the Gaussian distribution with mean value 1.5 and standard deviation 0.5 We use the same distribution for the candidates and the voters.

2. The *Small Uniform Candidates, Gauss Voters* model. We pick radii for the candidates uniformly at random from the range $[0.7, 1.2]$, and for the voters we use the same Gaussian, as in the previous model ($Gauss(1.5, 0.5)$).

3. The *Gauss Candidates, Small Uniform Voters* model. In this model we swap the distributions used in the previous one, that is, the candidates are picked from the Gaussian and the voters from the uniform distributions.

4. The *Small Uniform Candidates, Small Uniform Voters* model. We pick radii for both the candidates and the voters, uniformly at random from the range $[0.7, 1.2]$.

5. The *Large Uniform Candidates, Large Uniform Voters* model. We pick radii for both the candidates and the voters, uniformly at random from the range $[0, 3]$.

At first, it might be surprising why we don't include a constant radius, as one of the parameters. That is because, then all the profiles would be VR and CR (*COP*), as we show in Proposition 2.

Proposition 2. *If a profile P satisfies VCR and every agent (candidate/voter) in the profile, has the same constant radius, then P satisfies both VR and CR properties.*

Proof. Because :). □

Results

We present

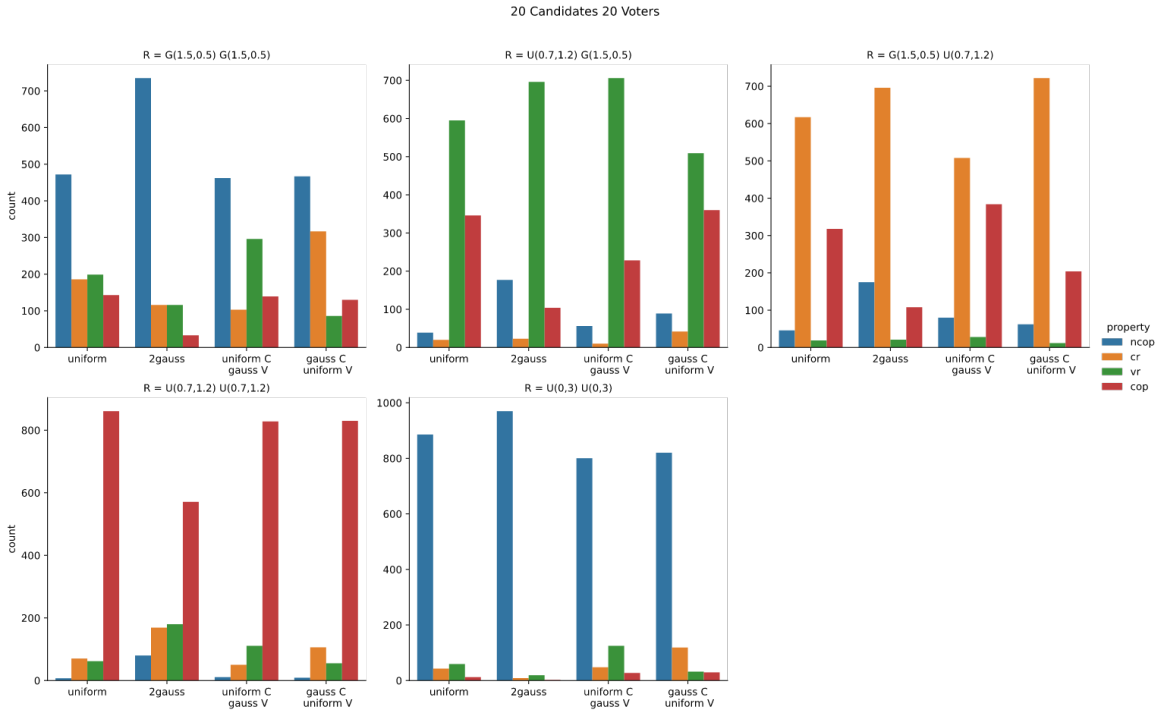

Figure 5.2 Temporary Image - Aggregated Statistics

5.3 Transforming VCR (NCOP) Profile into VR or CR

Chapter 6

Control Problem

6.1 Overview

6.1.1 AV-CC-DV for Single Peaked (VR = CI = row C1P = $r_c = 0$)

Algorithm from Faliszewski et al. (2009)

1. kandydat p na osi, pozostali na lewo i prawo
2. wybór przeciwników (groźny kandydat):
   (a) pierwszy na lewo c_L i pierwszy na prawo c_R takich, że $score(c_?) \geq score(p)$
   (b) powtarzamy dla kolejnych lewych c_{L_i} takich, że $score(c_{L_i}) > score(c_{L_{i-1}})$
   (c) analogicznie dla prawych c_{R_i} takich, że $score(c_{R_i}) > score(c_{R_{i-1}})$
3. usuwanie głosów - zaczynając od najbliższego prawego przeciwnika c_R (dla lewych - odbicie lustrzane)
   (a) notacja v: $b(v)$ początek przedziału (lewy) i $e(v)$ koniec przedziały (prawy)
   (b) wybieramy głosujących $\widehat{V}$ takich że $\{v \in V | \, b(v) \in (p, c_R]\}$
   (c) sortujemy $\widehat{V}$ malejąco po $e(v)$ (od najbardziej prawego końca przedziału)
   (d) usuwamy $scoreDelta(p, c_R) + 1$ głosów
4. powtarzamy do skutu, przeliczając aprobaty co iterację

6.1.2 AV-CC-DV for Candidate Range = VI = column C1P = $r_v = 0$

Algorithm from Magiera and Faliszewski (2014)

1. notacja CR dla c: $v_{f(c)}$ początek przedziału (lewy/górny) i $v_{e(c)}$ koniec przedziału (prawy/dolny)
2. kandydat p
3. Głosujący - wybieramy głosujących $V' = \{v \in V | \, p \notin A(v)\}$
4. Przeciwnicy - wybieramy kandydatów $\widehat{C} = \{c \in C \setminus \{p\} | \, scoreDelta(c, p) \geq 0\}$
5. Usuwanie głosów
   (a) zaczynając od przeciwnika c_N takiego, że $v_{e(c_N)}$ jest minimalne
   (b) usuwamy $deltaScore(c_N, p) + 1$ głosów, zaczynając od końca przedziału $v_{e(c_N)}$
6. powtarzamy do skutu, przeliczając V' i $\widehat{C}$ co iterację

6.2 AV-CC-DV for the VCR profiles

Notation used in the algorithm and later in the proofs:
x_a, r_a — point and radius
$b(a) = x_a - r_a$ — range beginning (left)
$e(a) = x_a + r_a$ — range end (right)
p — candidate chosen to be a winner
$\widehat{C} = \{c \in C \setminus \{p\} | \, scoreDelta(c, p) \geq 0\}$ — opponents (dangerous candidates)
$V' = \{v \in V | \, p \notin A(v)\}$ — potential voters to delete

We use “smart” greedy strategy for voter deletion:

1. We pick a leftmost opponent c_L that is the one with the minimal $b(c_L)$
2. We consider only voters who approve of c_L that is, $\widehat{V} = \{v \in V' | \, c_L \in A(v)\}$
3. We need to delete $deltaScore(c_L, p) + 1$ rightmost voters from $\widehat{V}$ that is the ones with the maximal $e(v)$, if it’s not possible we fail
4. We repeat the above steps until we succeed, recalculating $\widehat{C}$ and $\widehat{V}$ after each iteration

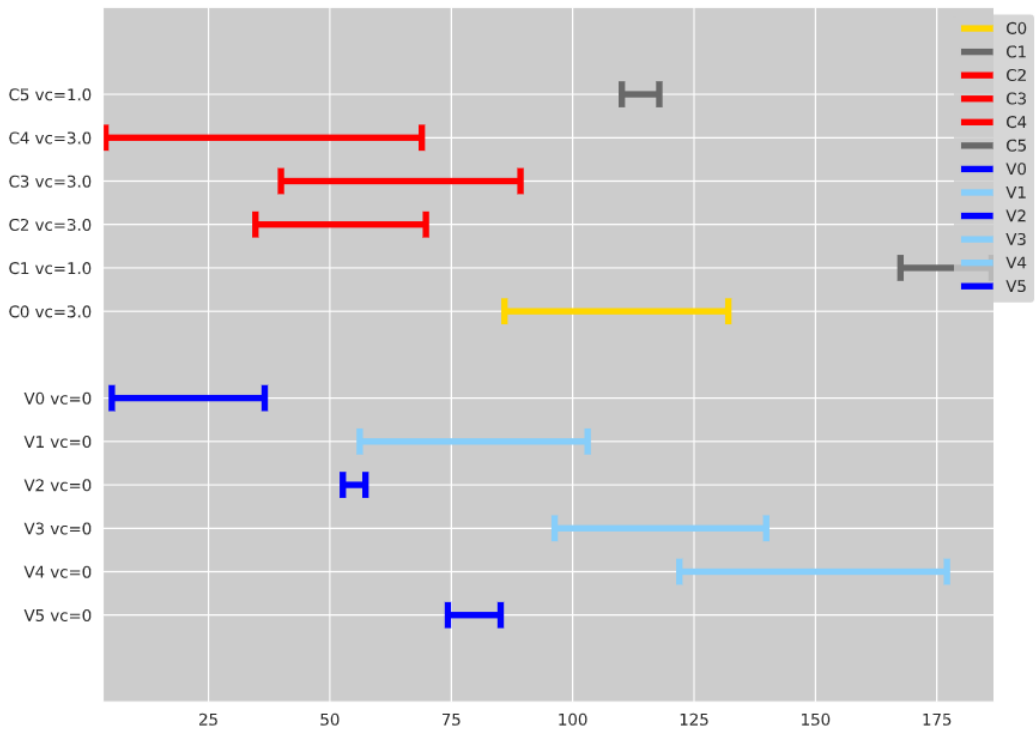

Figure 6.1 Temporary Image - VCR !C1P Election
yellow — candidate p
red — opponents
lightblue — voters whitelist
darkblue — possible voters to delete
grey — agents that can be ignored

6.2.1 Sketch Proof - CR case

We will show how the VCR algorithm is equivalent to the CR one from Magiera and Faliszewski (2014). Having voters in CR order, that is, they are still represented by a point x_v on the $\mathbb{R}$ line but have radius 0. Example in Figure 6.2.

Choosing opponents

In the CR algorithm, we choose candidates to process, ordered by $v_{e(c)}$ ascending. We can see that it is equivalent to picking the leftmost $(min(b(c)))$ candidate from the set of opponents $\widehat{C}$ in the VCR algorithm, because it will also have the minimal $v_{e(c)}$.

There is a special case of "contained candidate", for which the above statement is not true. In Section 6.2.3, we explain in detail why it is irrelevant for the overall correctness of the algorithm.

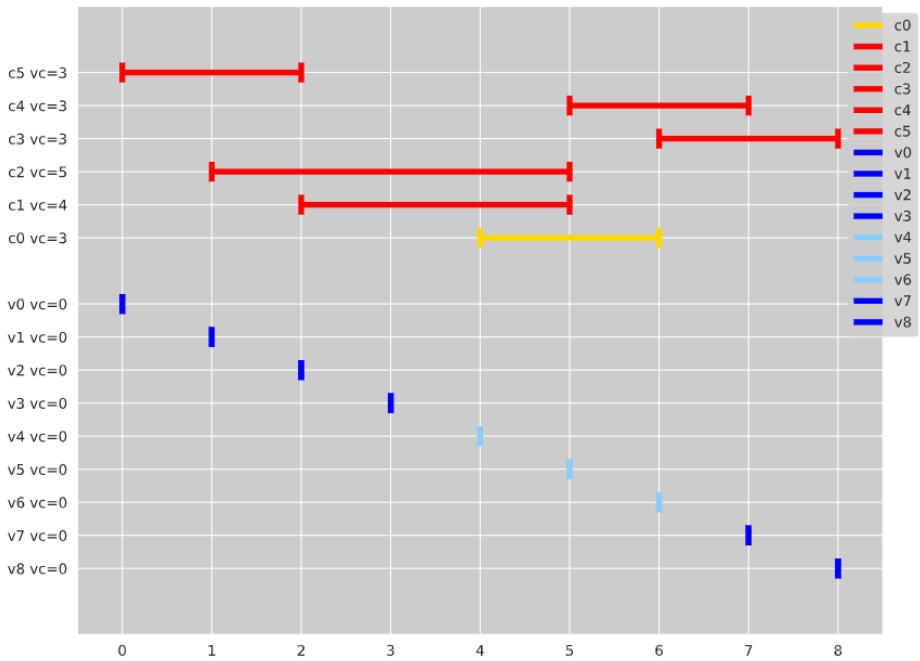

Figure 6.2 Temporary Image - CR Election

Voter deletion

Now, once we now how to pick an opponent c_L, we will show that choosing voters is equivalent as well. Notion of the beginning/end of the interval in the CR order is equivalent to the left/right end of the range in the VCR setting. Both algorithms try to delete $scoreDelta(p, c_L) + 1$ voters, CR one starts deleting voters from the end of interval, while VCR does virtually the same by deleting the rightmost voter first ($max(e(v))$).

6.2.2 Sketch Proof - VR case

We will show how the VCR algorithm is equivalent to the VR one from Faliszewski et al. (2009). Having candidates in VR order, that is, they are still represented by a point x_c on the $\mathbb{R}$ line but have radius 0. Example in Figure 6.3.

Choosing opponents

VR algorithm picks dangerous candidates to the left and right of candidate p, however later it proceeds only with defeating the right ones. Once finished with them, it does the "mirror flip" of the universe to process candidates on the left side. Even though the VCR algorithm picks leftmost candidate, we will show it is equivalent to the VR strategy under some basic assumptions.

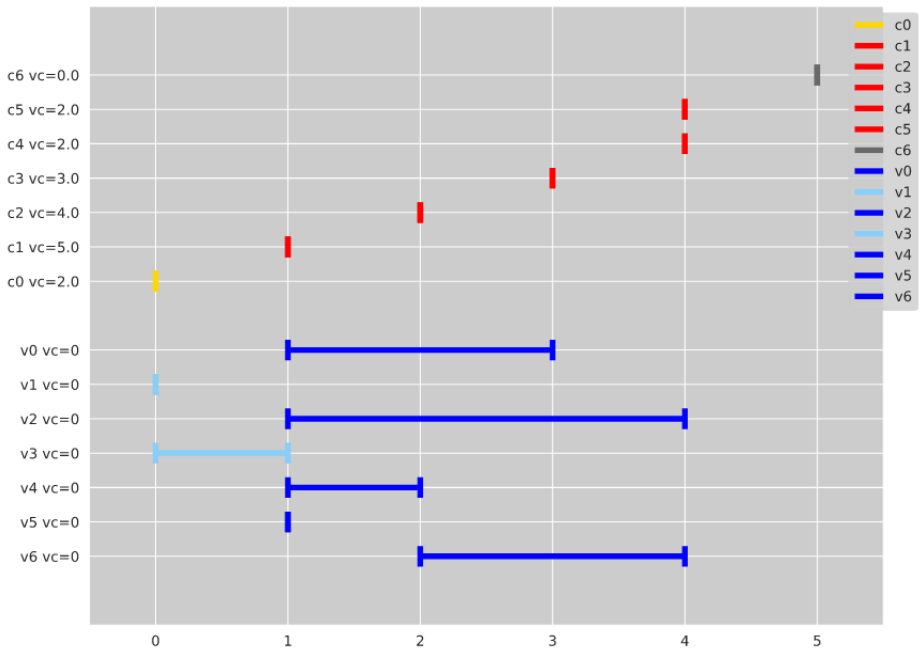

Figure 6.3 Temporary Image - VR Election

Following example in Figure 6.3, let's assume that candidate p always will be the first candidate in the VR order, which is equivalent to being the leftmost in the VCR order. Then we can see that the VCR algorithm strategy of choosing the leftmost candidate as an opponent will be the same as the VR algorithm choosing the first candidate on the right side of p. In the example it would be candidate c_1.

This assumption is correct because, having candidates in order $[c_L, p, c_R]$, if there was a voter v who approved of c_L and c_R it would also have to approve of p (because of the VR property), so we could not delete v anyway.

Now, expanding on the above, ...

(WIP: Faktycznie taki dowód nie jest w 100% przekonujący Mam 2 możliwości: 1) Wprowadzić krork "universe flip" do algorytmu i wtedy na pewno działa. 2) Zmienić argumentację i pokazać, że wybieranie najdalszego kandydat od p (najbardziej lewy po lewej stronie) też jest w porządku)

Voter deletion

The reasoning here is very similar to the one in the voter deletion part of the CR proof. Both algorithms try to delete $scoreDelta(c, p) + 1$ voters. VR algorithm starts deleting voters from the end of interval $(max(e(v)))$. VCR one does virtually the same by deleting the rightmost voter first.

6.2.3 Sketch Proof - VCR (not C1P) case

Now we will show why our solution is correct in the general VCR setting, even when the profile does not have VR/CR property, example in Figure 6.1. The proof is split into two stages, similarly to the previous ones. But first we will explain one special case of the election, that is worth examining.

Special case - contained candidates

In the VCR model it is possible that some candidates have their interval fully contained in the other's interval. Every voter who would approve the smaller candidate would also approve the larger one, however not vice versa. For this case, our strategy of choosing voters to delete might seem suboptimal, at first glance. In the minimal viable example from Figure 6.4, the algorithm would delete v_1 first, however, to defeat c_0, we need to delete 2 voters, so v_0 will be deleted as well.

The example illustrates why we do not have to worry about this special case. Every contained voter will have as much or less total votes than the larger candidate, he/she might not even be a dangerous candidate at all. Now we see that in the process of defeating the larger candidate we will automatically defeat the smaller one.

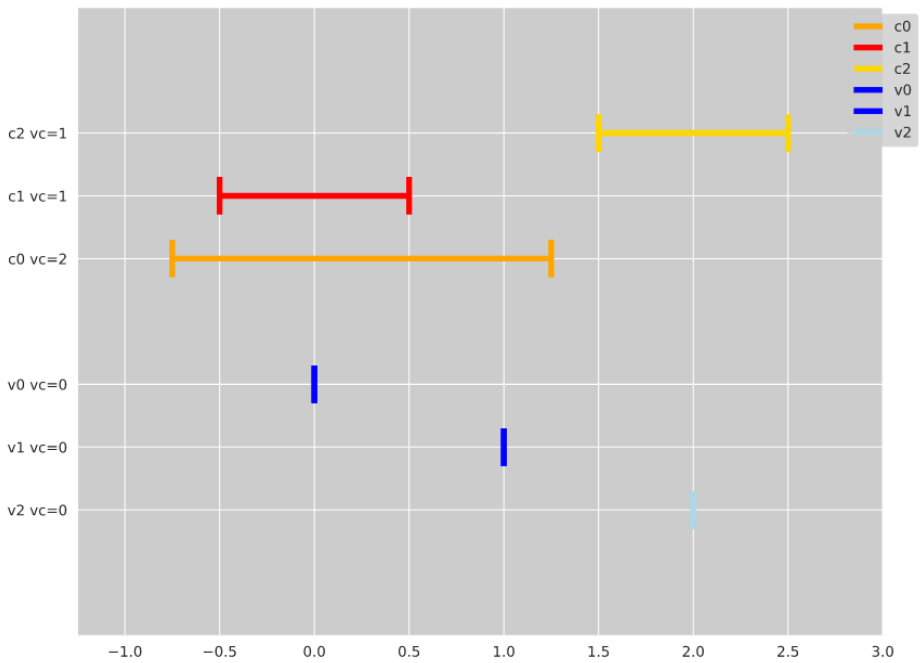

Figure 6.4 Temporary Image - Special Case

Choosing opponents

We consider only dangerous candidates (opponents), that is the ones with score higher or equal to the score of p. We process opponents one by one, starting with the one who has the leftmost beginning of the range, i.e., $min(b(c))$. After each iteration, we recompute the set of dangerous candidates. Thanks to this we are sure that all the steps we perform are necessary to find a solution (if it exists), hence optimal.

Voter deletion

Having currently processed opponent c_L, we consider only the voters who approve of him/her and do not approve of p, that is the voters whose range beginning is inside the candidate's range $\{v \in V' \mid b(v) \in [b(c_L), e(c_L)]\}$. We need to delete $scoreDelta(c_L, p) + 1$ voters to defeat the opponent. By picking the voters in order of the rightmost end of the range, i.e., $v_R = max(e(v))$, we try to maximize the number of other candidates who might be in the range $[b(v_R), e(v_R)]$. We don't have to worry about any candidate to the left, because of our leftmost candidate picking strategy, they are already defeated. This strategy ensures the solution is optimal if it exists.

6.3 AV-CC-AV for the VCR profiles

Our strategy is based on the solution from Magiera and Faliszewski (2014).

1. Add all $v \in W$ to the election
2. Is p the winner?
3. Try to delete $k' = |W| - k$ voters

Chapter 7

Evaluation

7.1 Conclusions

7.2 Future Work

Bibliography

Booth, K. S. and Lueker, G. S. (1976). Testing for the consecutive ones property, interval graphs, and graph planarity using pq-tree algorithms. *Journal of Computer and System Sciences*, 13(3):335 – 379.

Elkind, E. and Lackner, M. (2015). Structure in dichotomous preferences. *CoRR*, abs/1505.00341.

Faliszewski, P. (2021). An analysis of approval-based committee rules for 2d-euclidean elections.

Faliszewski, P., Hemaspaandra, E., Hemaspaandra, L. A., and Rothe, J. (2009). The shield that never was: Societies with single-peaked preferences are more open to manipulation and control. *CoRR*, abs/0909.3257.

Faliszewski, P., Skowron, P., Slink, A., and Talmon, N. (2017). *Multiwinner Voting: A New Challenge for Social Choice Theory*. AI Access.

Magiera, K. and Faliszewski, P. (2014). How hard is control in single-crossing elections? *Frontiers in Artificial Intelligence and Applications*, 263:579–584.